

如何讓其他人透過兩個代理式 AI (Antigravity + Claude) 使用 Onshape

雙代理協作與 API 成本優化實戰報告

日期：2026-08-29

目錄

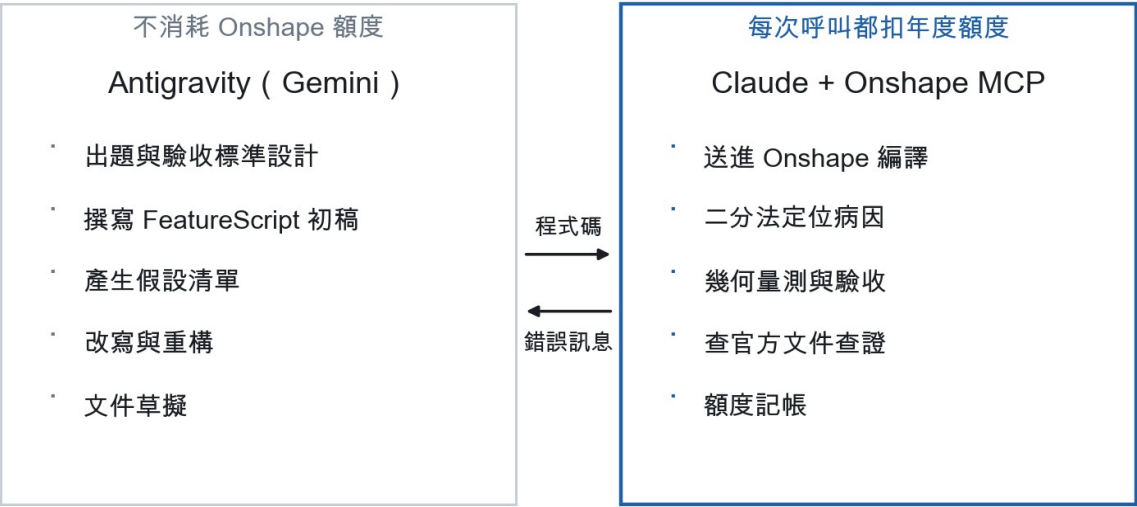
一、前言

如果你是「想用 AI 操作 Onshape 但還沒開始的人」，這份報告將為你避開無數的雷區。在這次實驗中，我們使用了兩個代理式 AI：Antigravity (Gemini) 負責出題與撰寫程式碼，且不吃 Onshape 額度；而 Claude 則負責送驗、量測與除錯。我們實作了一個名為 smartBoss 的自訂特徵，記錄了完整的除錯迴圈與真實的 API 呼叫成本。希望能藉由這份實錄，作為你踏入 Onshape AI 開發的避坑指南。

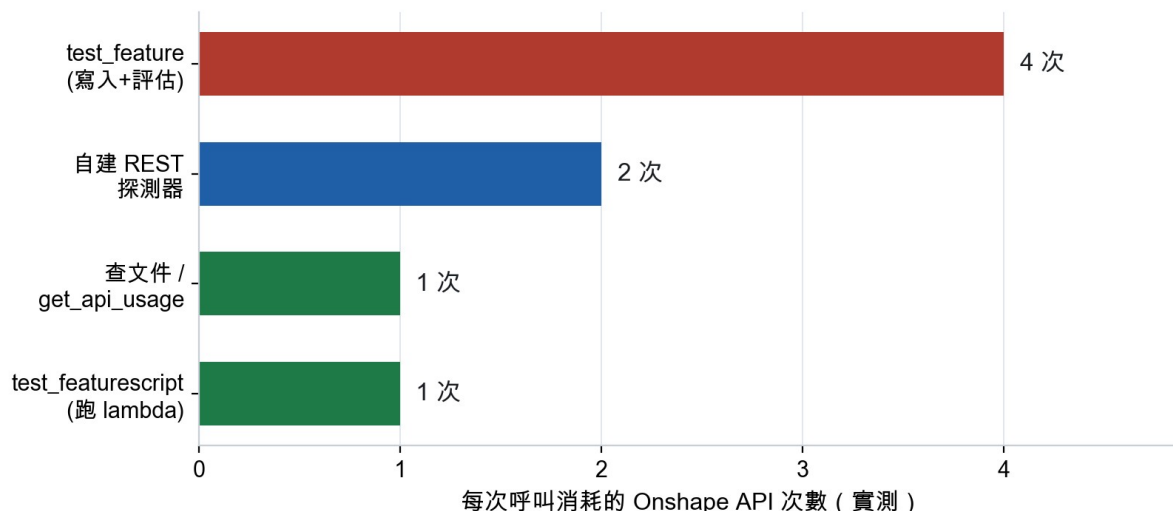
二、雙代理協作架構與 API 成本

在自動化過程中，我們必須精打細算每一次的 API 呼叫，因為 Onshape 的 API 額度有限 (預設 2,500 次，用完就會硬停，overageEnabled: false)。本輪消耗共 75 次 (含 19 次 MCP 工具呼叫與若干直接 REST 呼叫)。以下是我們實測得出的工具成本對照表：

工具	每次 API 呼叫數
test_featurescript (跑 lambda)	1
search_featurescript_documentation	1
get_api_usage	1
create_feature_studio	1
test_feature (寫進 Feature Studio 再評估)	4
自建探測器 (REST：PUT 內容 + GET featurespecs)	2



從實測中我們發現，除錯迴圈絕對不要用 test_feature。自建的 REST 探測器不僅只要一半成本，而且會回報「找到幾個特徵」，比 test_feature 那句沒有行號的錯誤訊息更有定位價值。



三、失敗路徑

在開發過程中，我們踩了許多坑。以下整理出的核心失敗路徑與教訓，詳細紀錄我們嘗試的做法、失敗原因，以及最終學到的經驗。

路徑一：依賴內建除錯工具與 LLM 診斷

嘗試：使用 test_feature 進行除錯，並請 Antigravity 診斷錯誤，當時模型給了 4 條言之鑿鑿的「確定錯誤」。

為何失敗：test_feature 每次消耗高達 4 次額度且不回報行號。而 LLM 對冷門 DSL (如 FeatureScript) 的標準函式庫記憶完全不可靠，其提出的 4 條診斷 (如 precondition 不能用 if、opFillet 不存在等) 全都是幻覺。

改用：建置自訂的 REST 探測器 (利用 PUT 內容 + GET featurespecs)，以特徵數是否大於 0 當作編譯成功的客觀訊號，並輔以二分法手動測試。

學到：標準函式庫的事實去查文件或做實驗，絕對不要問模型。除錯迴圈應使用自建探測器，不僅只要 2 次額度，還能獲得更有效的除錯訊號。

路徑二：幾何特徵驗證中的 API 隱藏限制

嘗試：在 geometry.fs 中使用 evBox3d 尋找特徵。

為何失敗：該符號在 geometry.fs 下不可見，並非參數給錯的問題，導致編譯持續失敗。諷刺的是，Antigravity 自己出題時點名應該用 evFaceTangentPlane，實作時卻沒有照做。

改用：將程式碼迴圈內逐項切斷驗證 (二分法)，找出真兇後改用 evFaceTangentPlane。

學到：即便 LLM 出題正確，實作時仍可能失誤，不能全盤信任。遇到編譯失敗時，採用二分實驗法能最快且精準地定位語法或可見性問題。

路徑三：布林運算的參數雷區

嘗試：在 opBoolean 進行 UNION (聯集) 時，將欲聯集的對象放入 targets 參數。

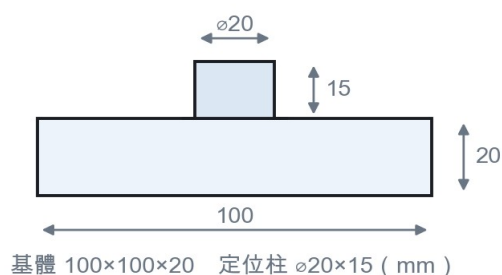
為何失敗：出現 @opBoolean: BOOLEAN_BAD_INPUT 錯誤。原以為是僅面接觸造成的干涉問題，於是將柱體下沉 1mm 製造干涉後測試，仍失敗。

改用：移除 targets 參數，將所有要聯集的物件全放入 tools 參數中。

學到：targets 參數僅用於 SUBTRACTION (交集) 或 INTERSECTION (差集)。幾何操作失敗有時是 API 定義問題，而非實體干涉。

四、幾何驗證

進入第二階段後，我們專注於驗證幾何結果。我們改用 test_featurescript 執行「建模 + 量測」合一的 lambda 函數。這一次呼叫僅需 1 次額度，比 create_geometry 造出實體再另外量測便宜得多。



驗收結果

體積	204712.389 mm ³	誤差 0.00000%
面數 (關圓角)	8	= 6 + 2N
面數 (開圓角)	9	= 6 + 3N
Part 數	1	布林聯集成功

五、總結與給後來者的操作準則

1. 版本標頭用 test_featurescript 回傳的 libraryVersion (目前 3070)，不要相信模型寫的版本號。
2. 除錯迴圈用自建 REST 探測器 (2 呼叫)，不要用 test_feature (4 呼叫且無行號)。
3. 不要相信 put_featurescript 的空輸出——它說明宣稱回傳 error，但實際對壞碼也回傳空字串。
4. 幾何驗證用 test_featurescript 跑「建模+量測」合一的 lambda，1 次呼叫就能拿到客觀數字。
5. 標準函式庫的事實去查文件或做實驗，不要問模型。兩次關鍵病因模型都沒說中，甚至還給了 4 條全錯的診斷。
6. 模型真正划算的用途：出題、寫初稿、產生假設清單——這些都不吃 Onshape 額度。

附錄：除錯歷程

輪次	動作	結果	學到什麼
1	Antigravity 產 v1，原樣送測	no features found	宣告 FeatureScript 2420，但實際 libraryVersion 是 3070
2	只改版本號 → v2	仍失敗	版本不是唯一病因，且錯誤訊息無行號
3	送最小空特徵當對照組	✅ 通過	harness 與版本都正常，問題在我們的程式碼
4	語法探針 (lambda 測 +=、append 等)	✅ 全部合法	排除三個假嫌疑
5	請 Antigravity 診斷	給了 4 條「確定錯誤」	全部是幻覺
6	用官方文件查證	opFillet 確實存在	不可採信單一模型對 std API 的記憶
7	put_featurescript 取編譯錯誤	回傳空字串	疑似無錯 → 但這是假訊號
8	故意送壞碼給 put_featurescript	仍回傳空字串	該工具說明宣稱回傳錯誤，實際不回報
9	自建 REST 探測器	可用	特徵數 0 = 沒編譯成功
10	二分：本體清空 / 只留迴圈	清空✅、只留迴圈❌	precondition 完全正常，if 與 Query 參數都合法
11	迴圈內逐項切	evaluateQuery✅ evPlane✅ evBox3d❌	找到真兇
12	evBox3d 加 tight 參數	仍❌	不是參數問題，是符號在 geometry.fs 下不可見
13	改 evFaceTangentPlane	✅	確定替代方案

14	寫 v4 (含 newSketchOnPlane 修正)	✅ 編譯通過，找到特徵	完成第一階段
15	完整流程 (含 opBoolean)	@opBoolean: BOOLEAN_BAD_INPUT	編譯過了，改成建模層級錯誤
16	印出布林前的中間狀態	輸入完全正確	問題在 opBoolean 本身
17	假設「僅面接觸」→ 柱體下沉 1mm	仍 BOOLEAN_BAD_INPUT	假設被推翻
18	UNION 時不給 targets，全放 tools	✅ 成功	真因：targets 只用於 SUBTRACTION / INTERSECTION
19	加圓角，量交界邊與面數	交界邊 1、面數 8→9	能正確追蹤新邊
20	修正寫回特徵 → v5	✅ 編譯通過	完成